

Khmelnyskyi National University
Department of Information Technonogy

Report

Laboratory work №3

Discipline: “Programming”

Topic: “*Programming of cyclic algorithms*”

Completed: 1st year student, group SE-26-1

Nykon Hrytsyshyn
Name, Surname

Checked:

Valeriy Martynyuk
Name, Surname

Purpose: To acquire skills in using loop operators, learn to program computational processes with a known number of repetitions, learn to program algorithms of an iterative cyclic structure.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, utils.h, main.c, runner.c, and utils.c), are provided in Appendices A...F.

Task 1

Tabulate the function $y = f(x)$ on the interval $[a; b]$ with a step $h = (b - a)/N$, where N is an integer (entered from the keyboard). The program should provide for the calculation of the values specified in the tasks.

Table 1 – Individual task

Variant	Function $y = f(x)$	Interval	
		a	b
4	$y = x(5 + x)$	1	6

The tabulation algorithm calculates function values across N equal step increments within the specified interval $[a; b]$. Safe user input parsing and range validation ($b > a, 1 \leq N \leq 1000$) are handled by auxiliary input utility functions `read_interval()` and `read_long_in_range()` (the full implementation of which is provided in Appendix F). A deterministic loop evaluates x_i and y_i at each iteration, outputting the formatted results in tabular form.

					LW-SE-26-1 09					
Ch.	Sheet	N° document	Signature	Date						
Developed		Hrytsyshyn N.B.			Laboratory work N°3			Letter	Sheet	Sheets
Checked		Martynyuk V.V.						H	1	32
N.control								KNU group SE-26-1		
Confirmed										

1.1. Block diagram of the program algorithm

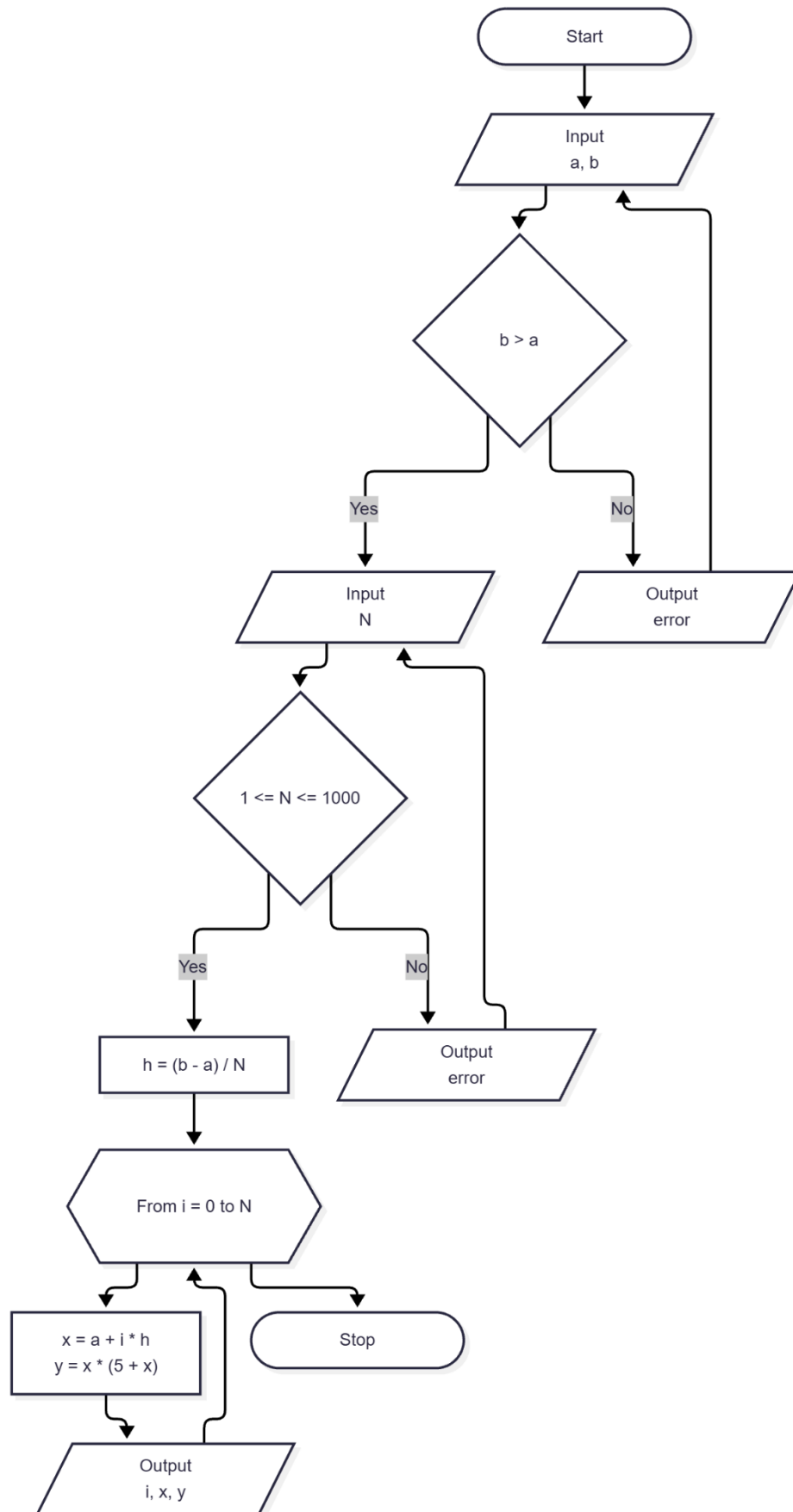


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

void run_task1(void) {
    double a = 0.0;
    double b = 0.0;
    long n = 0;

    puts("\n=== Task 1: tabulation of  $y = x(5 + x)$  on  $[a; b]$  ===\n");

    if (!read_interval(
        "Enter the interval start a: ", "Enter the interval end b: ", &a, &b
    )) {
        return;
    }

    if (!read_long_in_range(
        "Enter the number of steps N: ", 1, MAX_STEPS, &n
    )) {
        return;
    }

    const double h = (b - a) / (double)n;

    printf("\nh = (b - a) / N = %.6g\n\n", h);
    puts("   i |           x |           y");
    puts("-----+-----+-----");

    for (long i = 0; i <= n; ++i) {
        const double x = a + (double)i * h;
        const double y = x * (5.0 + x);

        printf("%4ld | %12.4f | %13.4f\n", i, x, y);
    }
}
```

1.3. Program results

```
=== Task 1: tabulation of y = x(5 + x) on [a; b] ===

Enter the interval start a: 1
Enter the interval end b: 6
Enter the number of steps N: 5

h = (b - a) / N = 1

  i |           x |           y
  ---+-----+-----
  0 |      1.0000 |      6.0000
  1 |      2.0000 |     14.0000
  2 |      3.0000 |     24.0000
  3 |      4.0000 |     36.0000
  4 |      5.0000 |     50.0000
  5 |      6.0000 |     66.0000
```

Fig. 2 – Task 1 results

Task 2

Develop a program to find the function y for a given x . Set the value of x independently, given that x is real (not an integer). Calculate the function y for five different values of x . Task option:

$$y = \frac{(x-1024)(x-1000)(x-996)\dots(x-2)}{(x-1)(x-3)(x-5)\dots(x-31)}, \quad (1)$$

To accurately compute the product of polynomial factors for five distinct real values of x without encountering floating-point overflow or underflow due to extremely large values, the algorithm introduces a custom normalization mechanism centered around a ScaledValue structure (storing a scientific mantissa and a base-10 exponent). The mathematical evaluation splits into two cyclic processes: a numerator loop iterating from 1000 down to 4 with a step of -4 (alongside boundary terms for 1024 and 2), and a denominator loop iterating from 1 to 31 with a step of $+2$. Terms are sequentially applied using dedicated multiply() and divide() functions that dynamically re-normalize the mantissa to maintain high numerical precision. Input validation for real numbers and division-by-zero edge case handling are supported by the I/O utilities detailed in Appendix F.

2.1. Block diagram of the program algorithm

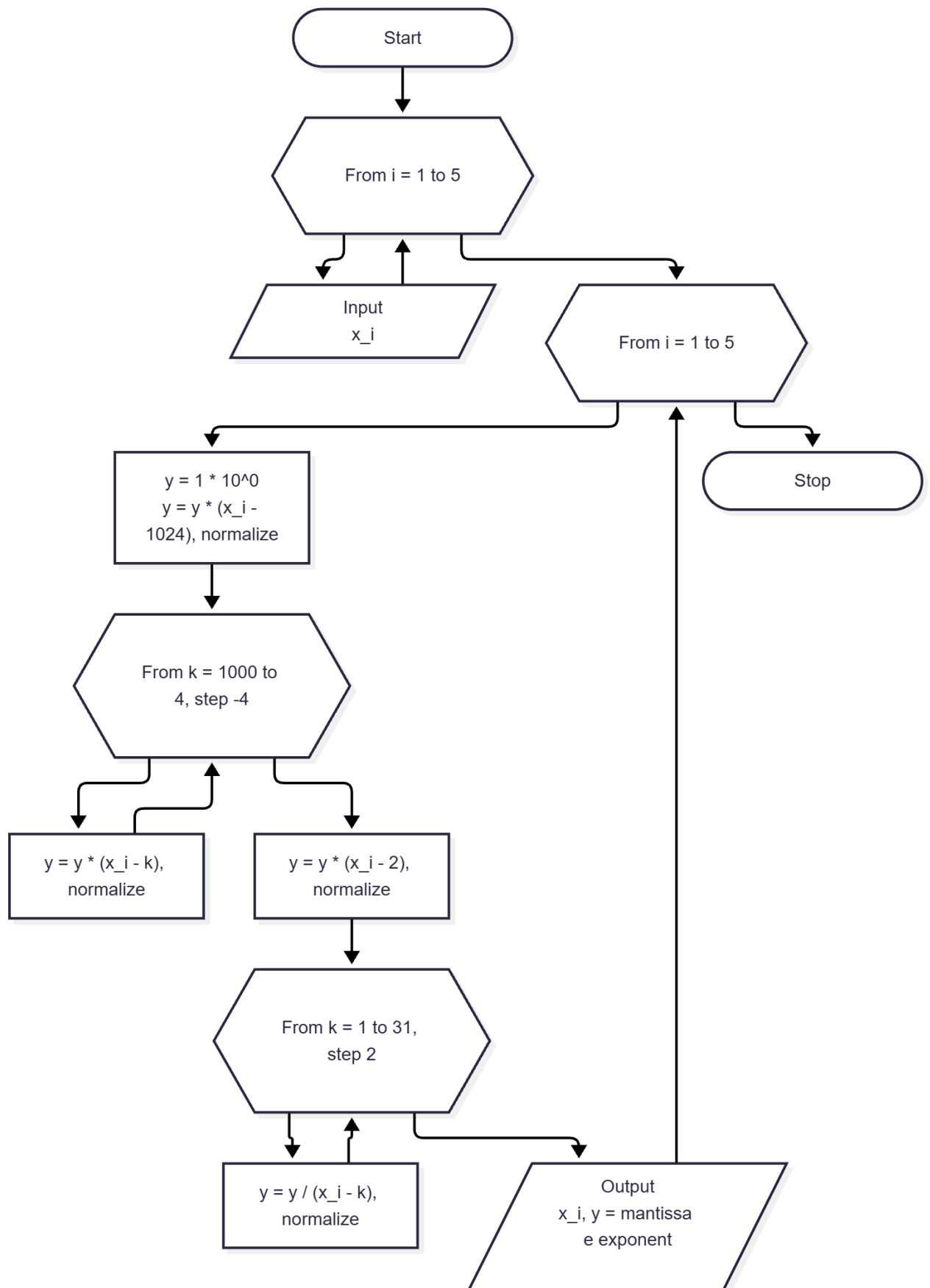


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#define X_COUNT 5
#define PROMPT_SIZE 32

#include "tasks.h"
#include "utils.h"

#include <math.h>
#include <stdio.h>

// Numerator: (x - 1024)(x - 1000)(x - 996)...(x - 4)(x - 2)
enum {
    NUM_FIRST = 1024,
    NUM_RUN_FROM = 1000,
    NUM_RUN_TO = 4,
    NUM_STEP = 4,
    NUM_LAST = 2
};

// Denominator: (x - 1)(x - 3)(x - 5)...(x - 31)
enum { DEN_FROM = 1, DEN_TO = 31, DEN_STEP = 2 };

// mantissa * 10^exp10
typedef struct {
    double mantissa;
    int exp10;
} ScaledValue;

static void normalize(ScaledValue *const value) {
    if (value->mantissa == 0.0) {
        value->exp10 = 0;
        return;
    }

    // Scale up when dropped below 1.0
    while (fabs(value->mantissa) < 1.0) {
        value->mantissa *= 10.0;
        value->exp10--;
    }

    // Scale down when reached or exceeded 10.0
    while (fabs(value->mantissa) >= 10.0) {
        value->mantissa /= 10.0;
        value->exp10++;
    }
}
```

```

    }
}

static void multiply(ScaledValue *const value, const double factor) {
    if (value->mantissa == 0.0) {
        return;
    }

    if (factor == 0.0) {
        value->mantissa = 0.0;
        value->exp10 = 0;
        return;
    }

    value->mantissa *= factor;
    normalize(value);
}

static void divide(ScaledValue *const value, const double factor) {
    if (value->mantissa == 0.0) {
        return;
    }

    if (factor == 0.0) {
        value->mantissa = NAN;
        value->exp10 = 0;
        return;
    }

    value->mantissa /= factor;
    normalize(value);
}

static ScaledValue calculate_y(const double x) {
    ScaledValue y = {.mantissa = 1.0, .exp10 = 0};

    multiply(&y, x - NUM_FIRST);

    for (int k = NUM_RUN_FROM; k >= NUM_RUN_TO; k -= NUM_STEP) {
        multiply(&y, x - k);
    }

    multiply(&y, x - NUM_LAST);
}

```

```

    for (int k = DEN_FROM; k <= DEN_TO; k += DEN_STEP) {
        divide(&y, x - k);
    }

    return y;
}

static void print_scaled(const ScaledValue y) {
    if (isnan(y.mantissa)) {
        puts("undefined (division by zero)");
        return;
    }

    if (y.mantissa == 0.0) {
        puts("+0.000000e+0");
        return;
    }

    printf(
        "%c%.6fe%+d\n", y.mantissa < 0.0 ? '-' : '+', fabs(y.mantissa), y.exp10
    );
}

void run_task2(void) {
    puts("\n=== Task 2: product of factors for five values of x ===\n");
    puts("y = (x-1024)(x-1000)(x-996)...(x-2) / ((x-1)(x-3)...(x-31))\n");

    double xs[X_COUNT];
    char prompt[PROMPT_SIZE];

    for (int i = 0; i < X_COUNT; ++i) {
        snprintf(prompt, sizeof(prompt), "Enter real x%d: ", i + 1);

        if (!read_double_in_range(
            prompt, -MAX_INPUT_ABS, MAX_INPUT_ABS, &xs[i]
        )) {
            return;
        }
    }

    puts("");
}

```

```

for (int i = 0; i < X_COUNT; ++i) {
    printf("x = %12.4f -> y = ", xs[i]);
    print_scaled(calculate_y(xs[i]));
}
}

```

2.3. Program results

```

=== Task 2: product of factors for five values of x ===

y = (x-1024)(x-1000)(x-996)...(x-2) / ((x-1)(x-3)...(x-31))

Enter real x1: 0.5
Enter real x2: 2.75
Enter real x3: 15.5
Enter real x4: 500.25
Enter real x5: 1500.5

x =      0.5000  ->  y = +1.590622e+629
x =      2.7500  ->  y = +6.845860e+628
x =     15.5000  ->  y = +1.608161e+625
x =    500.2500  ->  y = +1.649069e+528
x =   1500.5000  ->  y = +1.044564e+700

```

Fig. 4 – Task 2 results

Task 3

Write a program according to the option:

- 4) Given an integer N . Find the sum of the digits of this number.

The algorithm computes the sum of the decimal digits of an integer N by iteratively extracting its least significant digit using modulo arithmetic ($\% 10$) and reducing the remaining value via integer division ($/ 10$). To seamlessly handle negative input values and avoid two's complement overflow edge cases (such as `LONG_MIN`), the input is converted to its absolute magnitude using an unsigned long container prior to processing. An iterative while loop continues digit accumulation until the magnitude is reduced to zero. Safe non-overflowing integer reading and input validation are provided by the utility functions detailed in Appendix F.

3.1. Block diagram of the program algorithm

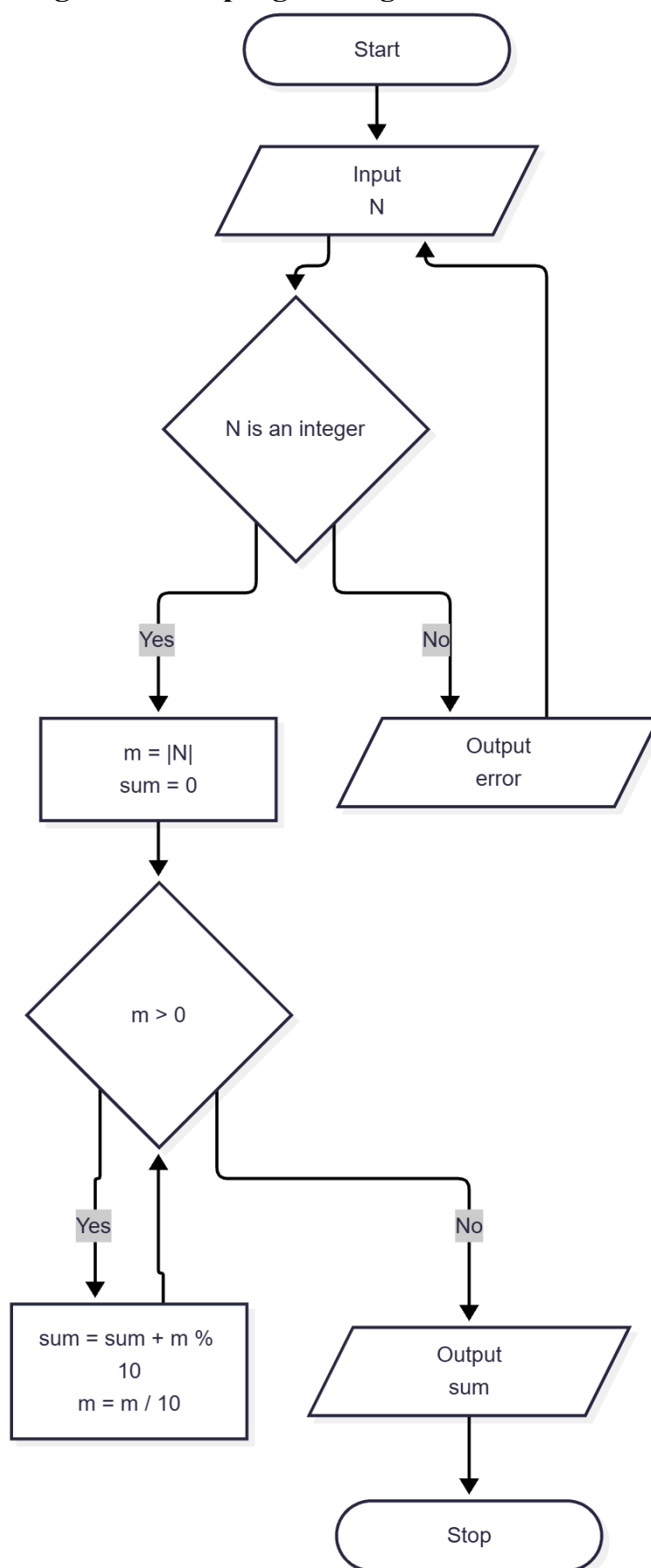


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <limits.h>
#include <stdio.h>

static int sum_of_digits(unsigned long magnitude) {
    int sum = 0;

    while (magnitude > 0) {
        sum += (int)(magnitude % 10);
        magnitude /= 10;
    }

    return sum;
}

void run_task3(void) {
    long n = 0;

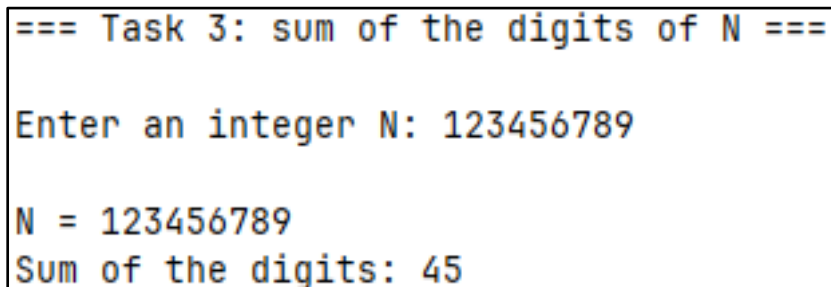
    puts("\n=== Task 3: sum of the digits of N ===\n");

    if (!read_long_in_range("Enter an integer N: ", LONG_MIN, LONG_MAX, &n))
    {
        return;
    }

    const unsigned long magnitude =
        n < 0 ? 0UL - (unsigned long)n : (unsigned long)n;

    printf("\nN = %ld\n", n);
    printf("Sum of the digits: %d\n", sum_of_digits(magnitude));
}
```

3.3. Program results



```
=== Task 3: sum of the digits of N ===

Enter an integer N: 123456789

N = 123456789
Sum of the digits: 45
```

Fig. 6 – Task 3 results

Task 4

Tabulate the function $y = f(x)$ on the interval $[a; b]$ with a step $h = (b - a)/N$, where N is an integer (entered from the keyboard). The program provides for the calculation of the values indicated in the data.

Table 2 – Individual task

Variant	Function $y = f(x)$	Interval		Calculated values
		a	b	
4	$y = \arctg(x)$	0	2	Arithmetic average $0 < y < 0,5$

The program tabulates the inverse trigonometric function $y = \arctan(x)$ over a closed interval $[a; b]$ divided into N subintervals with a step size $h = \frac{b-a}{N}$, while dynamically accumulating statistical data for function values that satisfy a user-defined threshold range $lower < y < upper$.

The execution logic is structured into three main phases:

- Input Acquisition & Strict Range Validation: Interval boundaries $[a; b]$, total steps N ($1 \leq N \leq 1000$), and output bounds $(lower, upper)$ are ingested and validated using custom I/O helper routines (`read_interval()` and `read_long_in_range()`, the full implementation of which is provided in Appendix F). This ensures mandatory structural constraints, such as $b > a$ and $upper > lower$, before commencing computation.
- Cyclic Evaluation & Conditional Filtering: A deterministic for loop iterates i from 0 to N , computing the argument $x_i = a + i \cdot h$ and evaluating $y_i = \arctan(x_i)$ via `atan()`. Each iteration outputs i , x_i , and y_i in formatted tabular structure. Concurrently, if $lower < y_i < upper$, y_i is added to an accumulator sum and the valid entry counter 'count' is incremented.
- Statistical Post-Processing: After completing loop execution, a conditional check evaluates count. If no tabulated values satisfied the range condition (count == 0), an appropriate notification message is displayed. Otherwise, the algorithm computes the arithmetic mean $\bar{y} = \frac{sum}{count}$ and displays the result alongside the total number of matched values.

4.1. Block diagram of the program algorithm

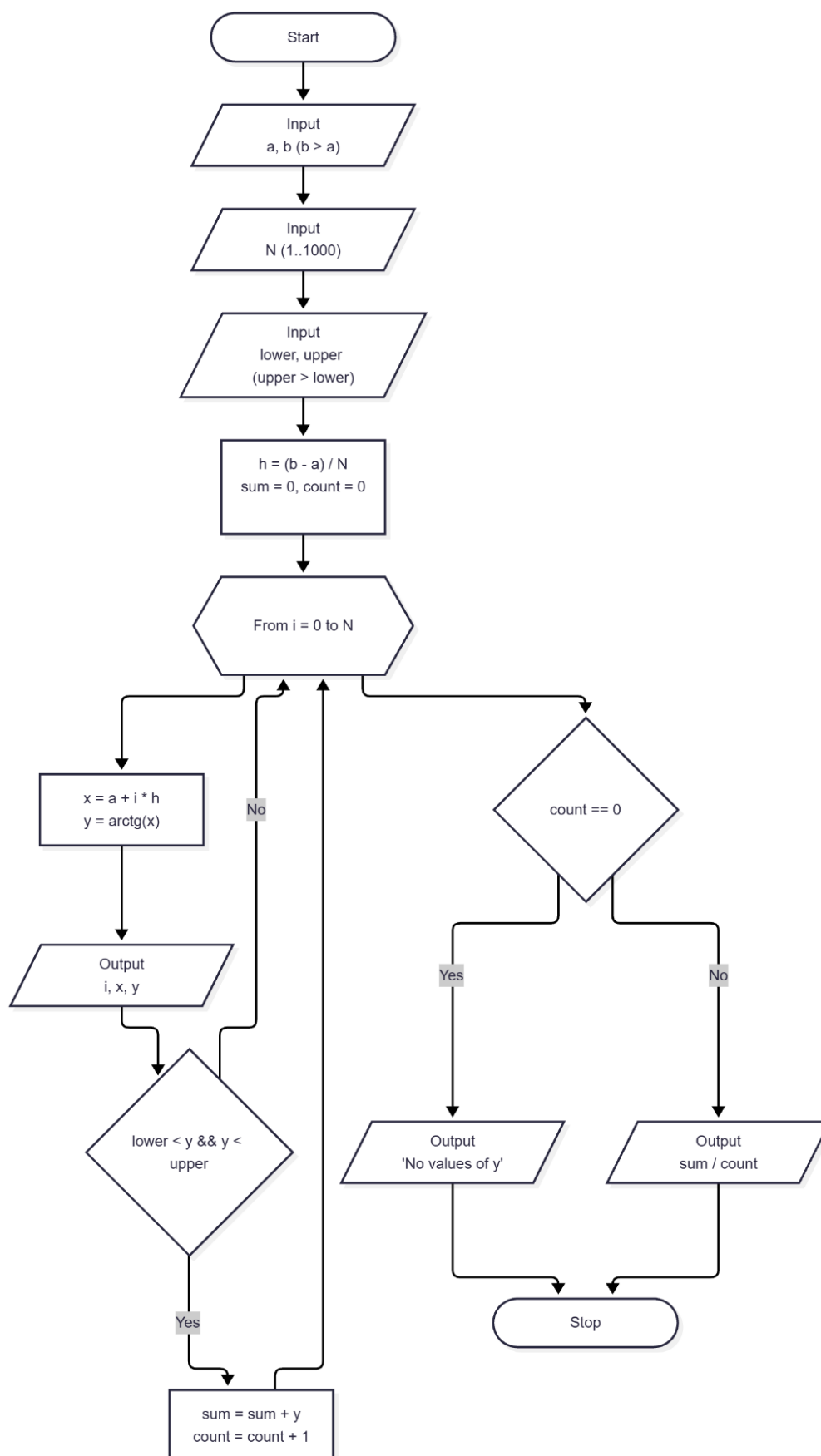


Fig. 7 – Block diagram of the program algorithm

4.2. Program text

```
#include "tasks.h"
#include "utils.h"
#include <math.h>
#include <stdio.h>

void run_task4(void) {
    double a = 0.0;
    double b = 0.0;
    double lower = 0.0;
    double upper = 0.0;
    long n = 0;

    puts("\n=== Task 4: tabulation of  $y = \arctg(x)$  on  $[a; b]$  ===\n");

    if (!read_interval(
        "Enter the interval start a: ", "Enter the interval end b: ", &a, &b
    )) {
        return;
    }

    if (!read_long_in_range(
        "Enter the number of steps N: ", 1, MAX_STEPS, &n
    )) {
        return;
    }

    if (!read_interval(
        "Enter the lower bound of y: ", "Enter the upper bound of y: ",
        &lower, &upper
    )) {
        return;
    }

    const double h = (b - a) / (double)n;
    double sum = 0.0;
    long count = 0;

    printf("\nh = (b - a) / N = %.6g\n\n", h);
    puts(" i |      x |      y");
    puts("-----+-----+-----");

    for (long i = 0; i <= n; ++i) {
        const double x = a + (double)i * h;
```

```

const double y = atan(x);
printf("%4ld | %12.4f | %8.4f\n", i, x, y);

if (y > lower && y < upper) {
    sum += y;
    ++count;
}

puts("");

if (count == 0) {
    printf("No values of y satisfy %g < y < %g.\n", lower, upper);
    return;
}

printf(
    "Arithmetic mean of y with %g < y < %g: %.4f (%ld values)\n",
    lower,
    upper,
    sum / (double)count,
    count
);
}

```

4.3. Program results

```

=== Task 4: tabulation of y = arctg(x) on [a; b] ===

Enter the interval start a: 0
Enter the interval end b: 2
Enter the number of steps N: 5
Enter the lower bound of y: 0
Enter the upper bound of y: 0.5

h = (b - a) / N = 0.4

  i |          x |          y
  --+-----+-----
  0 |    0.0000 |    0.0000
  1 |    0.4000 |    0.3805
  2 |    0.8000 |    0.6747
  3 |    1.2000 |    0.8761
  4 |    1.6000 |    1.0122
  5 |    2.0000 |    1.1071

Arithmetic mean of y with 0 < y < 0.5: 0.3805 (1 values)

```

Fig. 8 – Task 4 results

Task 5

Calculate the sum of the series $\sum_n^\infty$ with accuracy = 0.0001. According to the compiled program, perform calculations at the given value of x and output the number of terms that were summed up.

Table 3 – Individual task

Variant	a_n	k	x
4	$(-1)^{n-1} \frac{x^{2n-1}}{2n-1}$	0	0,65

The overall execution pipeline is structured as follows:

- **Input Parameters & Boundary Validation:** The algorithm prompts for the initial index k ($0 \leq k \leq 1000$), real argument x ($|x| \leq 1.0$), and target accuracy ϵ . Input processing and range validation are handled via custom I/O helper functions (`read_long_in_range()`, `read_double_in_range()`, and `read_x()`, detailed in Appendix F). When $k = 0$, the first term contains $x^{-1} = \frac{1}{x}$, so $x = 0$ is explicitly rejected to prevent division by zero.
- **Recurrence Scheme & Iterative Evaluation:** To optimize computational efficiency and avoid calling `pow()` inside the loop, initial state variables are established prior to iteration: $\text{power} = x^{2k-1}$ and $\text{sign} = (-1)^{k-1}$. An iterative loop evaluates $a_n = \frac{\text{sign} \cdot \text{power}}{2n-1}$ for $n = k, k+1, \dots$. On each step, sign flips its sign ($\text{sign} = -\text{sign}$), power is multiplied by x^2 ($\text{power} *= x_squared$), terms is incremented, and a_n is added to sum.
- **Convergence Termination:** The iteration terminates as soon as the absolute value of the current term falls below the specified threshold ($|a_n| < \epsilon$).
- **Analytical Verification:** For initial indices $k \leq 1$, the program verifies numerical results against exact closed-form expressions using `atan(x)`. Specifically, when $k = 1$, the Taylor series converges to `arctan(x)`, whereas for $k = 0$, the check accounts for the leading term and compares against `arctan(x) + \frac{1}{x}`.

5.1. Block diagram of the program algorithm

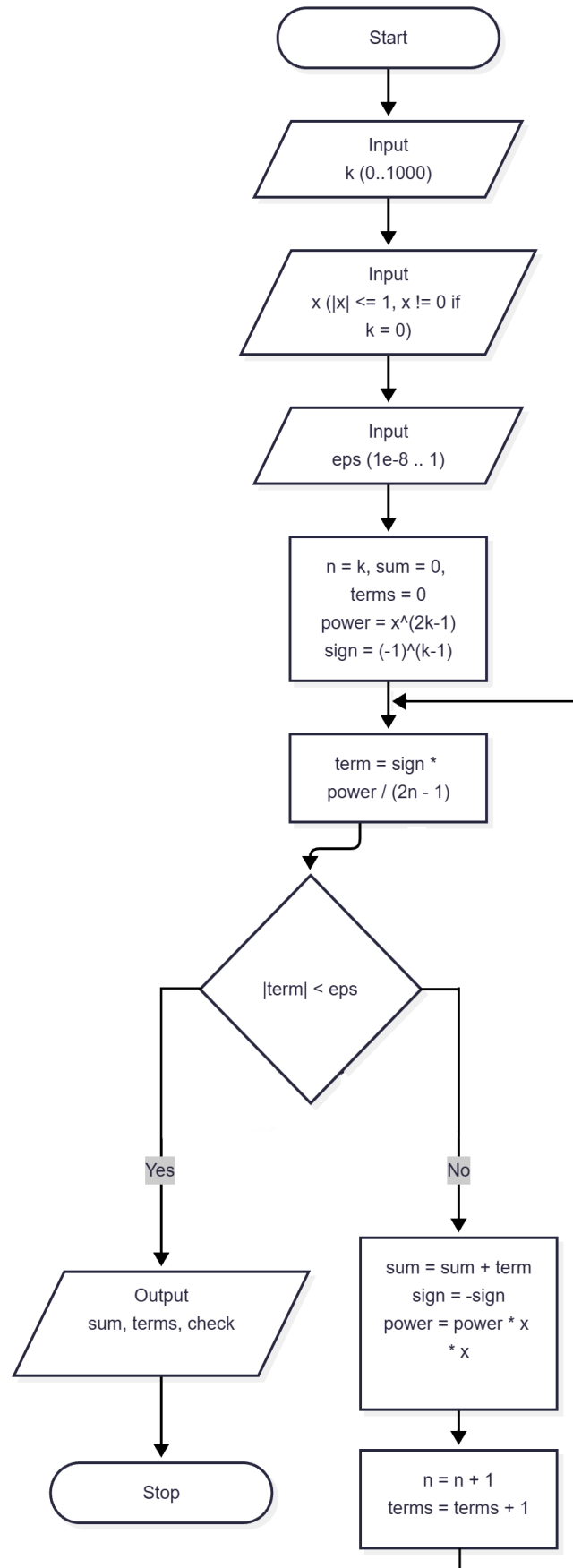


Fig. 9 – Block diagram of the program algorithm

5.2. Program text

```
#define MAX_ABS_X 1.0

#define MIN_EPSILON 1e-8
#define MAX_EPSILON 1.0

#define MIN_START_INDEX 0
#define MAX_START_INDEX 1000

#include "tasks.h"
#include "utils.h"

#include <math.h>
#include <stdbool.h>
#include <stdio.h>

static bool read_x(const long k, double *const out_x) {
    while (true) {
        if (!read_double_in_range(
            "Enter x ( $|x| \leq 1$ ): ", -MAX_ABS_X, MAX_ABS_X, out_x
        )) {
            return false;
        }

        if (k != 0 || *out_x != 0.0) {
            return true;
        }

        puts("Error: x must not be 0 when k = 0 (the first term is 1 / x).");
    }
}

void run_task5(void) {
    long k = 0;
    double x = 0.0;
    double epsilon = 0.0;

    puts("\n=== Task 5: sum of a series with a given accuracy ===\n");
    puts("a_n =  $(-1)^{(n-1)} * x^{(2n-1)} / (2n-1)$ , sum over n = k, k+1, ... \n");

    if (!read_long_in_range(
        "Enter the first index k: ", MIN_START_INDEX, MAX_START_INDEX,
        &k
    )) {
    }
```

```

    return;
}

if (!read_x(k, &x)) {
    return;
}

if (!read_double_in_range(
    "Enter the accuracy epsilon: ", MIN_EPSILON, MAX_EPSILON, &epsilon
)) {
    return;
}

const double x_squared = x * x;
double sum = 0.0;
long terms = 0;

double power = pow(x, 2 * k - 1); //  $x^{(2n-1)}$ 
double sign = k % 2 == 0 ? -1.0 : 1.0; //  $(-1)^{(n-1)}$ 

for (long n = k;; ++n) {
    const double term = sign * power / (double)(2 * n - 1);

    if (fabs(term) < epsilon) {
        break;
    }

    sum += term;
    sign = -sign;
    power *= x_squared;
    ++terms;
}

printf("\nk = %ld, x = %g, epsilon = %g\n", k, x, epsilon);
printf("Sum of the series: %.6f\n", sum);
printf("Number of summed terms: %ld\n", terms);

if (k <= 1) {
    printf(
        "Check, %s: %.6f\n",
        k == 0 ? "arctg(x) + 1/x" : "arctg(x)",
        atan(x) + (k == 0 ? 1.0 / x : 0.0)
    );
}
}

```

5.3. Program results

```
=== Task 5: sum of a series with a given accuracy ===  
  
a_n = (-1)^(n-1) * x^(2n-1) / (2n-1), sum over n = k, k+1, ...  
  
Enter the first index k: 0  
Enter x (|x| <= 1): 0.65  
Enter the accuracy epsilon: 0.0001  
  
k = 0, x = 0.65, epsilon = 0.0001  
Sum of the series: 2.114809  
Number of summed terms: 9  
Check, arctg(x) + 1/x: 2.114837
```

Fig. 10 – Task 5 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in designing and implementing branched algorithms using conditional statements. The software implementation successfully met all requirements across the four tasks, including coordinate quadrant determination, age group classification, piecewise function computation, and numeric digit extraction.

APPENDIX A

Block diagram of the main program control flow algorithm

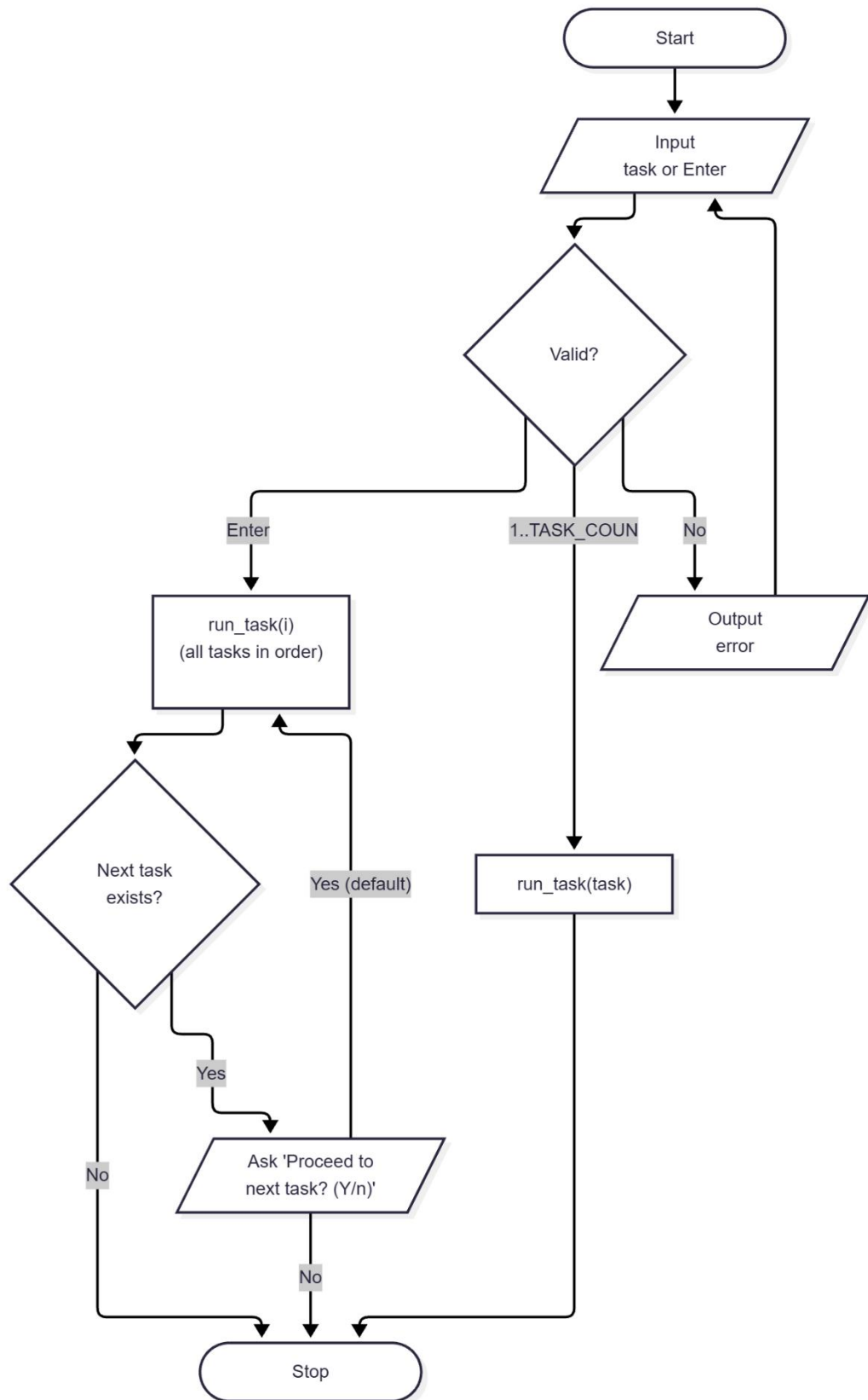


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3) X(4) X(5)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

// Max number of steps N accepted by the tabulation tasks
#define MAX_STEPS 1000

// Max absolute value of a real number accepted from the keyboard
#define MAX_INPUT_ABS 1e9

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);

bool read_long_in_range(
    const char *prompt,
    long min, long max,
    long *out_value
);
bool read_double_in_range(
    const char *prompt,
    double min, double max,
    double *out_value
);
bool read_interval(
    const char *prompt_min, char *prompt_max,
    double *out_min, double *out_max
);
bool read_optional_long_in_range(
    const char *prompt,
    long min, long max,
    long *out_value,
    bool *out_provided
);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX D

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdbool.h>
#include <stdio.h>

#define PROMPT_SIZE 96

int main(void) {
    puts(
        "=====\\n"
        " Laboratory work 3: Cyclic algorithms (variant 9)\\n"
        "=====\\n"
    );

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt,
        sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks [ALL]: ",
        TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task, &provided))
    {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX E

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d? (Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX F

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr, void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }
        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
    const size_t count,
```

```

void *const outs[],
const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }
}

```

```

void *outs[MAX_READ_VALUES];

va_list args;
va_start(args, count);

for (size_t i = 0; i < count; ++i) {
    outs[i] = va_arg(args, double *);
}

va_end(args);

return read_line_values(prompt, count, outs, parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs, parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {
    while (true) {
        if (read_long(prompt, out_value)) {

```

```

        if (*out_value >= min && *out_value <= max) {
            return true;
        }

        printf("Error: value must be between %ld and %ld.\n", min, max);
    } else if (feof(stdin)) {
        return false;
    } else {
        puts("Invalid input. Please enter an integer.");
    }
}
}

bool read_double_in_range(
    const char *prompt,
    const double min,
    const double max,
    double *out_value
) {
    while (true) {
        if (read_double(prompt, out_value)) {
            // Written this way so that NaN is rejected as well
            if (*out_value >= min && *out_value <= max) {
                return true;
            }

            printf("Error: value must be between %g and %g.\n", min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter a number.");
        }
    }
}

bool read_interval(
    const char *prompt_min,
    const char *prompt_max,
    double *out_min,
    double *out_max
) {
    if (!read_double_in_range(
        prompt_min, -MAX_INPUT_ABS, MAX_INPUT_ABS, out_min
    )) {
        return false;
    }
}

```

```

    }

    while (true) {
        if (!read_double_in_range(
            prompt_max, -MAX_INPUT_ABS, MAX_INPUT_ABS, out_max
        )) {
            return false;
        }

        if (*out_max > *out_min) {
            return true;
        }

        printf("Error: the upper bound must be greater than %g.\n", *out_min);
    }
}

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press Enter.");
        } else if (value < min || value > max) {

```

```

        printf("Error: value must be between %ld and %ld.\n", min, max);
    } else {
        *out_value = value;
        *out_provided = true;
        return true;
    }
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL && !feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {

```

```

const char *start = buffer;
while (isspace((unsigned char)*start)) {
    start++;
}

size_t len = strlen(start);
while (len > 0 && isspace((unsigned char)start[len - 1])) {
    len--;
}

if (len == 0) {
    return default_answer;
}

if (equals_ignore_case(start, len, "y") ||
    equals_ignore_case(start, len, "yes")) {
    return true;
}

if (equals_ignore_case(start, len, "n") ||
    equals_ignore_case(start, len, "no")) {
    return false;
}

printf("Please answer 'y' or 'n'.\n");
}
}

```